

Model Training Methods & Validation

Chapter 02 · Episode 01 · Domain 1.1 — Supervised, Unsupervised, RL & TEVV



Learning Objectives

1. Identify the three training paradigms and their cyber applications
2. Name the primary vulnerability of each training method
3. Explain TEVV as the mandatory pre-deployment gate

Supervised Learning

Trains on labeled data (known correct output)

Malware classifiers, spam filters, IDS

The label is the ground truth — and the attack surface

Attack: label poisoning



Unsupervised Learning

Trains on unlabeled data; discovers structure

Anomaly detection & network baselines

'Normal' = whatever it sees most in training

Attack: baseline poisoning (slow / boiling-frog)



Reinforcement Learning

An agent learns from reward signals

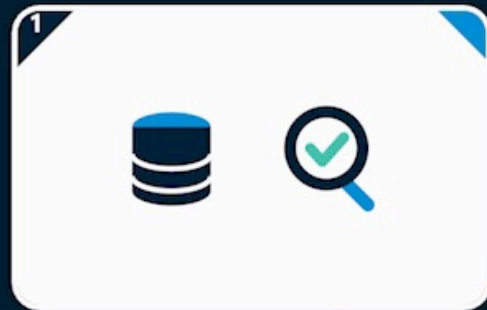
Autonomous pentest, adaptive IDS

Attack: reward manipulation

Plus excessive-agency risk — human oversight critical

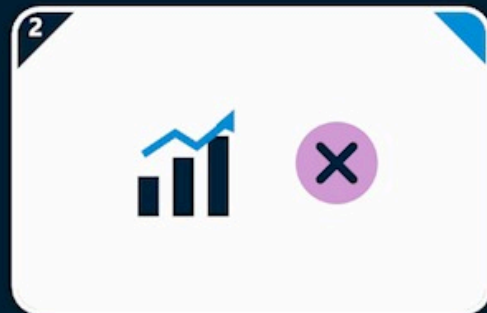


TEVV — The Pre-Deployment Gate



1 TEST

Accuracy on held-out set;
false-negative rate



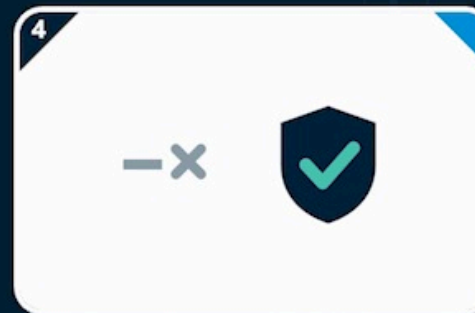
2 EVALUATE

Bias, fairness &
subgroup performance



3 VERIFY

Meets documented requirements



4 VALIDATE

Safe under adversarial &
edge inputs

The Training Pipeline Is Infrastructure

Preprocessing, checkpoints, CI/CD = attack surfaces

Supply-chain attacks inject poison data or weights

...without touching the model interface

Controls: version control + reproducibility

PIPELINE IS THE SURFACE



TRAINING DATA

SURFACE



PREPROCESSING

SURFACE



CHECKPOINTS

SURFACE



BUILD PIPELINE

SURFACE



THE SHIPPED MODEL

Poison enters upstream; the interface never changes.

1 Supervised Pipeline Check

PASS

2 Unsupervised Baseline Audit

WARN

3 RL Reward Signal Analysis

PASS

4 TEVV Gate Validation

FAIL

HOW TO RUN

WINDOWS

```
python training_pipeline_audit.py
```

MACOS/LINUX

```
python3 training_pipeline_audit.py
```

TRY IT Change the TEVV adversarial_robustness score to 85 and observe the pipeline completing successfully.

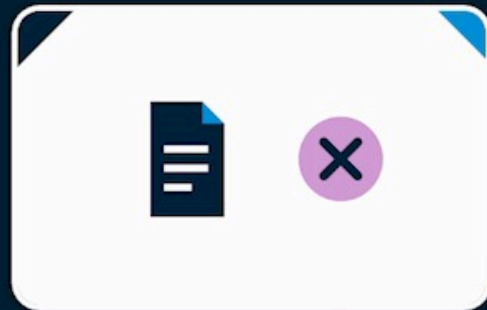
`training_pipeline_audit.py`

pure Python 3.10+, no external libraries

Demo: Training Pipeline Security Audit

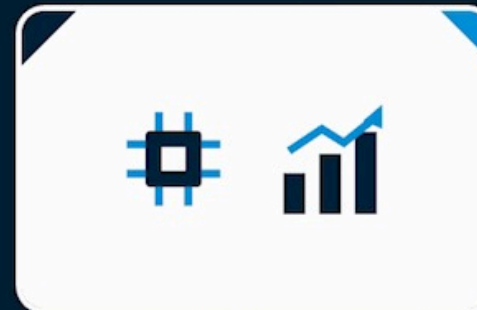
Supervised, unsupervised & RL stages — flags label poisoning, baseline drift, reward manipulation

Training Method Reference



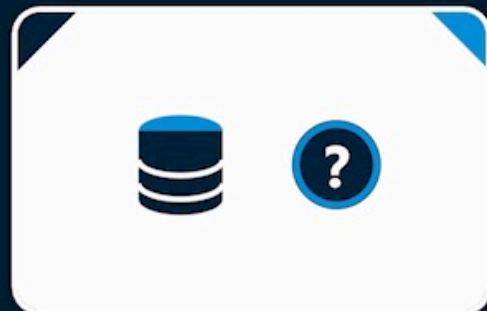
SUPERVISED

Labeled → label poisoning



REINFORCEMENT

Reward → manipulation
+ agency



UNSUPERVISED

Unlabeled → baseline
poisoning



ALL THREE

TEVV before deploy;
version control

Exam Match Rule

Mislabeled training data → supervised

Slow normalization of malice →
unsupervised baseline

Unexpected autonomous action → RL
excessive agency

TEVV gates every model before production



Fine-Tuning, Epochs, Pruning & Quantization

Chapter 02 · Episode 02 · Domain 1.1 — The Optimization Layer



Learning Objectives

1. Explain fine-tuning, epochs, pruning, and quantization
2. Identify the security tradeoff each optimization introduces
3. Apply the rule: re-run TEVV after any optimization step

Fine-tuning —

data in the weights

Adapt a capable base model on domain data

Incident reports tune a threat assistant

Traffic patterns tune an anomaly detector

Training data is encoded into the weights

WHERE FINE-TUNING DATA GOES



BASE MODEL



INCIDENT
REPORTS



TRAFFIC
PATTERNS



PII + SECRETS

FINE-TUNED MODEL WEIGHTS

DOMAIN PATTERNS

SENSITIVE DATA



EXTRACTABLE BY INVERSION

Fine-tune only on data classified at or below
the model's deployment sensitivity level.

Two fine-tuning risks

Inversion extracts what the model learned
Fine-tune at or below deployment level
Poisoned samples implant new behaviors
The base model tests clean
Audit fine-tuning data separately

THE TRANSFER LEARNING ATTACK



SAMPLE



SAMPLE



POISONED



FINE-TUNING PASS



ADAPTED MODEL

IMPLANTED BEHAVIOR

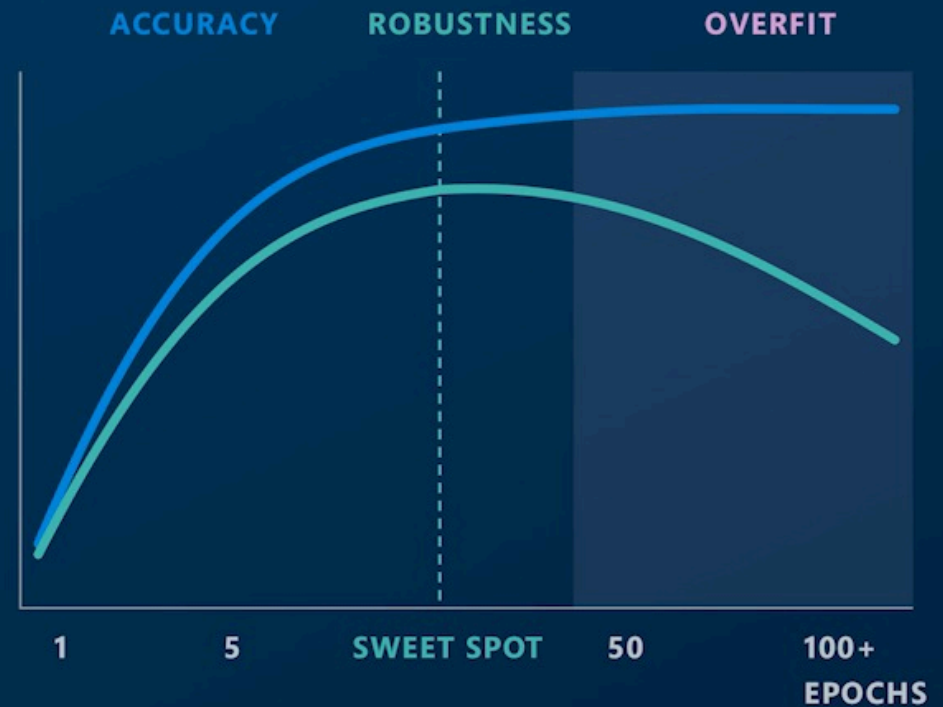
BASE MODEL STILL TESTS CLEAN

Audit the fine-tuning dataset separately —
base-model testing never sees the implant.

Epochs — the training curve

One epoch — one full pass through the data
Weights adjust a little with every pass
Early epochs improve both curves
Accuracy plateaus; robustness can degrade

THE EPOCH TRAINING CURVE



Both improve early — then accuracy plateaus while robustness degrades. TEVV catches it.

Underfitting versus overfitting

UNDERFITTING

Too few epochs

Patterns not learned reliably

Misses subtle attack variants

OVERFITTING

Too many epochs

Memorizes instead of generalizing

Sensitive examples become extractable

The more dangerous failure mode

Overfit models leak their training data

Inversion attacks recover training examples

Held-out test sets catch memorization

Cross-validation confirms generalization

TEVV is the required gate before deployment



Pruning —

near-zero weights trimmed away

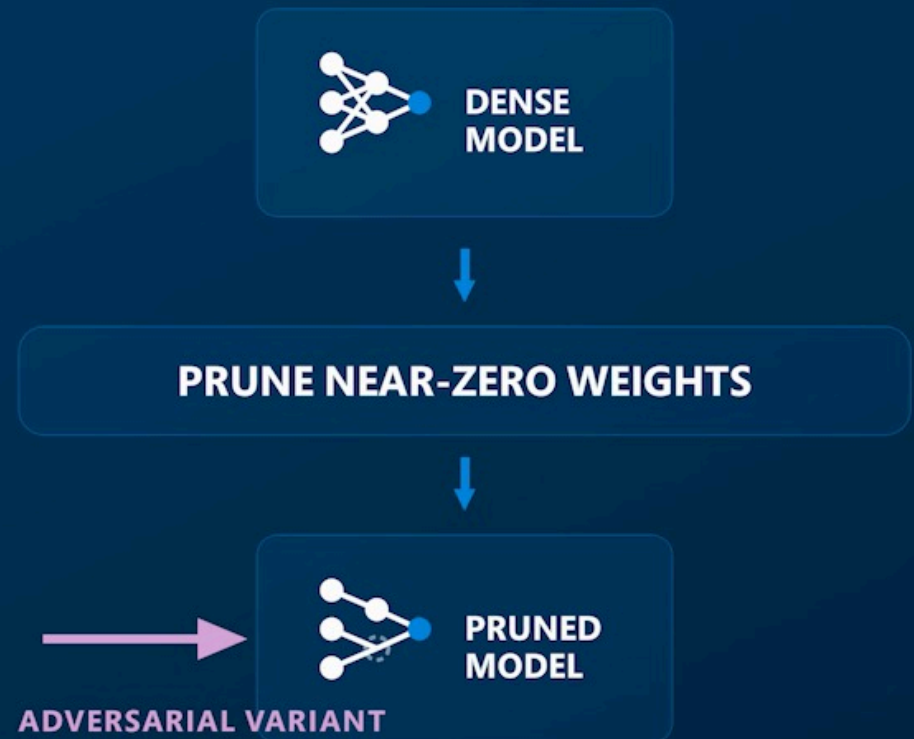
Smaller and faster, similar accuracy

Trimmed parameters handled the edge cases

Adversarial inputs target pruned regions

A pruned classifier misses crafted variants

WHAT PRUNING TRIMS AWAY



The trimmed parameters handled the edge cases —
re-run adversarial TEVV after every prune.

Quantization —

32-bit precision down to 8-bit

About 4× smaller; runs on edge hardware

Precision loss blurs fine distinctions

On-device weights are physically exposed

Theft and reverse engineering follow



What compression does to the model

Size

Shrinks 50–75%

Speed

Runs 2–4× faster

Accuracy

Holds about the same on typical inputs

Robustness

Reduced — varies by model

Edge risk

New — on-device model theft becomes possible

AFTER PRUNING + QUANTIZATION

SIZE DOWN 50–75%

SPEED UP 2–4×

ACCURACY ABOUT THE SAME

ROBUSTNESS REDUCED — VARIES

EDGE RISK NEW — THEFT POSSIBLE



TREAT IT AS A NEW MODEL

Previous TEVV results do not carry over — re-run the full suite before redeploying.

The core principle —

an optimized model is a new model

Every step trades robustness for efficiency

Previous TEVV results do not carry over

Re-run the full TEVV suite after any change

Include adversarial robustness testing



Edge deployment risks



THEFT

Weights on the device can be pulled and reverse-engineered.



EVASION

Crafted inputs exploit the precision that quantization lost.



TAMPERING

A backdoored checkpoint replaces the real one in storage.

Optimization technique reference



FINE-TUNING

Data in weights — classify it, test for inversion.



EPOCHS

Overfit memorizes and underfit misses — TEVV both.



PRUNING

Robustness drops — adversarial re-test.



QUANTIZATION

Precision loss and on-device theft — adversarial TEVV.

- 1 Baseline evaluation
- 2 Fine-tune — three epochs
- 3 Fine-tune — twelve epochs · overfit
- 4 Prune 40% of parameters
- 5 Quantize to 8-bit
- 6 TEVV re-check + hardening

... HOW TO RUN ...

WINDOWS

```
python optimization_tradeoff_analyzer.py
```

MACOS/LINUX

```
python3 optimization_tradeoff_analyzer.py
```

TRY IT

Change the robustness threshold to 50 — what happens to the risk alert?

```
optimization_tradeoff_analyzer.py  
pure Python 3.10+ · no external libraries
```

The optimization tradeoff analyzer

A pure-Python simulation of robustness lost and restored

Exam tips



FINE-TUNING

Sensitive data lives in the weights — classify first.



EPOCHS

Overfitting means memorization means extraction.



PRUNE + QUANT

Robustness drops — rerun adversarial TEVV.



THE RULE

Treat every optimized model as a brand-new model.